Kombinatorická optimalizace (B4M35KO)

1 Celočíselné lineární programování (ILP)

Celočíselné lineární programování je matematická optimalizační metoda, kde všechny nebo některé proměnné musí nabývat celočíselných hodnot. Oproti klasickému LP je prostor řešení nespojitý a nekonvexní. Problém ILP je obecně **NP-těžký**.

1.1 Modelování a logické podmínky

Klíčovým nástrojem pro formulaci logických závislostí je binární proměnná $y \in \{0, 1\}$ a **metoda velkého M** (kde M je dostatečně velká konstanta, reprezentující např. teoretickou horní mez).

- **Implikace ($x_1 \implies x_2$):** Pokud je vybrána možnost 1, musí být vybrána i možnost 2.
  Model: $x_1 \leq x_2$.
- **Exkluzivní disjunkce (XOR):** Právě jedna z možností.
  Model: $x_1 + x_2 = 1$.
- **Disjunkce (Alespoň jedno ze dvou omezení platí):** $A \leq B \vee C \leq D$.
  Model: Zavedeme binární proměnnou y. $A \leq B + M \cdot y \quad \wedge \quad C \leq D + M \cdot (1 - y)$.
- **Fixní náklady (Setup cost):** Stroj stojí fixně K a má kapacitu M.
  Model: Produkce $x \leq M \cdot y$, účelová funkce $\min(\cdots + K \cdot y)$.

Zkouška: Pokročilé ILP modely

- **Absolutní hodnota (např. $\min |x_1 - x_2|$):** LP neumí absolutní hodnotu. Zavedeme spojitou proměnnou $z \geq 0$ a účelovou funkci $\min z$. Přidáme dvě omezení: $x_1 - x_2 \leq z$ a $-(x_1 - x_2) \leq z$.
- **Nespojité (skokové) funkce:** *Cena je $7 + 5x$ pokud $x > 0$, a 0 pokud $x = 0$.*
  Zavedeme $y \in \{0, 1\}$. Omezení: $x \leq M \cdot y$. Účelová funkce: $\min(7y + 5x)$.
- **Minimalizace rozdílu extrémů (Min-Max difference):** *Rozdělte lup tak, ať je rozdíl mezi nejbohatším a nejchudším co nejmenší.*
  Zavedeme spojité P_{max}, P_{min}. Pro každý lup L_j platí: $P_{max} \geq L_j$ a $P_{min} \leq L_j$. Minimalizujeme $(P_{max} - P_{min})$.

1.2 Totálně unimodulární matice (TUM)

Věta (Schrijver): Pokud je matice omezení A totálně unimodulární a pravé strany b jsou celočíselné, pak každý vrchol polyedru $Ax \leq b$ je celočíselný. LP relaxace vyřešená simplexovou metodou tedy přímo najde optimální celočíselné řešení (v polynomiálním čase).

Matice A je TUM, pokud determinant každé její čtvercové submatice je $0, 1$ nebo -1. **Postačující podmínky TUM:**

1. Všechny prvky jsou z $\{0, 1, -1\}$. Každý sloupec obsahuje nanejvýš jednu $+1$ a nanejvýš jednu -1 (např. matice incidence pro sítě).

2. **Rozdělené řádky (Bipartitní graf):** Prvky z $\{0, 1\}$. Každý sloupec má max. dvě jedničky. Řádky lze rozdělit do dvou disjunktních množin tak, že v každém sloupci s dvěma jedničkami je jedna v první a druhá ve druhé množině řádků.

3. **Vlastnost po sobě jdoucích jedniček:** Prvky z $\{0, 1\}$. V každém sloupci tvoří jedničky souvislý blok (lze toho dosáhnout permutací řádků).

1.3 Metoda Branch & Bound (B&B)

Algoritmus kombinující prohledávání a efektivní ořezávání stavového prostoru.

- **Relaxace a Bounding (Omezování):** Vyřešíme LP relaxaci problému. Její výsledek tvoří horní (u maximalizace) nebo dolní (u minimalizace) mez.
- **Branching (Větvení):** Vybereme proměnnou x_k, která vyšla neceločíselně (hodnota v). Rozdělíme na dva podproblémy: $x_k \leq \lfloor v \rfloor$ a $x_k \geq \lceil v \rceil$.
- **Pruning (Eliminace uzlu):** Uzel je zahozen pokud:
  1. LP relaxace je nepřípustná.
  2. Hodnota LP relaxace je horší než hodnota aktuálně nejlepšího celočíselného řešení (Incumbent).
  3. Řešení LP relaxace je plně celočíselné (případně aktualizujeme Incumbent).

2 Nejkratší cesty (Shortest Paths - SPT)

Problém nalezení nejkratší cesty z počátečního uzlu s do cílového t (nebo všech ostatních). Využívá **Bellmanův princip optimality**: Každá podcesta nejkratší cesty je sama o sobě nejkratší cestou mezi svými koncovými uzly. Z toho plyne **Bellmanova rovnice**: $l(w) = \min_v\{l(v) + c(v, w)\}$.

2.1 Algoritmy

- **Dijkstra:** Časová složitost $O(m + n \log n)$ (s Fibonacciho haldou). **Vyžaduje nezáporné váhy hran.** Jedná se o *label-setting* algoritmus (jakmile je uzel vybrán z prioritní fronty, jeho vzdálenost je finální).
- **Bellman-Ford:** Časová složitost $O(n \cdot m)$. **Zvládá záporné váhy**. Jedná se o *label-correcting* algoritmus (vzdálenosti se mohou měnit až do konce). Pokud v n-té iteraci (vnější smyčka) dojde k relaxaci nějaké hrany, graf obsahuje **záporný cyklus**.
- **Floyd-Warshall:** $O(n^3)$. Dynamické programování pro všechny dvojice uzlů (APSP). Matice l_{ij}^k představuje délku nejkratší cesty z i do j používající pouze vnitřní uzly z množiny $\{1, \ldots, k\}$.
- **DAG (Acyklické grafy):** V topologickém uspořádání lze nejkratší cestu najít v čase $O(n+m)$.

Zkouška: Důkazy a transformace SPT

- **Důkaz korektnosti Dijkstrova algoritmu (Indukcí):**
  - *Předpoklad:* Graf nemá záporné hrany. Množina permanentních uzlů je R.
  - *Báze:* Pro $R = \{s\}$ je $l(s) = 0$ optimální.
  - *Indukční krok:* Nechť R obsahuje uzly s optimální vzdáleností. Algoritmus vybere uzel $v \notin R$ s minimálním $l(v)$. Jakákoliv jiná cesta do v by musela opustit množinu R přes nějaký uzel $k \notin R$. Délka takové cesty by byla $l(k) + c(k \ldots v)$. Z našeho výběru víme, že $l(v) \leq l(k)$. Protože neexistují záporné hrany ($c(k \ldots v) \geq 0$), platí $l(v) \leq l(k) + c(k \ldots v)$. Žádná kratší cesta tedy neexistuje a $l(v)$ je optimální.

- **Maximalizace spolehlivosti (Probabilities):** Hrany mají spolehlivost $p_i \in (0, 1]$. Hledáme cestu maximalizující $\prod p_i$.
  Řešení: Použijeme logaritmus. $\max \prod p_i \iff \min \sum(-\log p_i)$. Váhy nastavíme na $c_e = -\log(p_i)$. Protože $p_i \leq 1$, je $\log p_i \leq 0$, takže $c_e \geq 0$. Můžeme použít Dijkstru.

- **Časově expandované sítě (Constrained Shortest Path):** *Najděte nejkratší cestu (vzdálenost c_{ij}), jejíž čas trvání $t_{ij} \in \mathbb{Z}^+$ nepřekročí T.*
  Řešení: Vytvoříme $T + 1$ vrstev grafu (jedna pro každý časový okamžik). Hranu (u, v) vedeme z uzlu u v čase t do uzlu v v čase $t + t_{uv}$ s váhou c_{uv}. Cílem je najít nejkratší cestu z $(s, 0)$ do libovolného (t, t_{end}), kde $t_{end} \leq T$.

3 Síťové toky (Network Flows)

Síť je definována orientovaným grafem s kapacitami $u(e)$, dolními mezemi $l(e)$, zdrojem s a cílem t. **Kirchhoffův zákon** vyžaduje zachování toku v každém uzlu (mimo s a t). **Max-Flow Min-Cut teorém:** Maximální tok ze zdroje do cíle je roven kapacitě minimálního řezu oddělujícího s a t. Z maticové formulace (TUM) navíc plyne **Teorém o integralitě toku**: jsou-li kapacity celočíselné, existuje celočíselný maximální tok.

3.1 Algoritmy pro maximální tok

- **Ford-Fulkerson (FF):** Hledá zlepšující (augmenting) cestu v *reziduálním grafu.*
  - *Dopředná hrana:* Reziduální kapacita $u_f(e) = u(e) - f(e)$.
  - *Zpětná hrana:* Reziduální kapacita $u_f(e) = f(e) - l(e)$. Umožňuje "vrátit"tok a poslat jej jinudy.

  Složitost je $O(|E| \cdot F)$ (kde F je hodnota max toku). Pro iracionální kapacity nemusí skončit.

- **Edmonds-Karp:** Vylepšení FF, které vybírá vždy **nejkratší** zlepšující cestu (dle počtu hran, pomocí BFS). Silně polynomiální složitost $O(|V| \cdot |E|^2)$.

- **Minimální řez:** Množina uzlů A dosažitelných ze zdroje s v reziduálním grafu po zastavení algoritmu definuje minimální řez. Kapacita řezu je $\sum_{e \in \delta^+(A)} u(e) - \sum_{e \in \delta^-(A)} l(e)$.

Zkouška: Transformace sítí a Min-Cost Flow

- **Počáteční přípustný tok s nenulovými dolními mezemi ($l(e) > 0$):** (Velmi časté zadání)
  1. Předefinujeme kapacity: $u'(e) = u(e) - l(e)$. (Tím posuneme dolní mez na 0).
  2. Převedeme dolní meze na bilanci uzlů: pro každý uzel v spočteme $b(v) = \sum_{in} l(e) - \sum_{out} l(e)$.
  3. Přidáme super-zdroj s' a super-cíl t'.
  4. Pro uzly s $b(v) > 0$ přidáme hranu (s', v) s kapacitou $b(v)$.
  5. Pro uzly s $b(v) < 0$ přidáme hranu (v, t') s kapacitou $-b(v)$.
  6. Z původního cíle t do zdroje s přidáme zpětnou hranu s kapacitou ∞.
  7. Spustíme Max-Flow ze s' do t'. Pokud nalezený tok plně nasytí všechny hrany vycházející ze s', pak počáteční přípustný tok pro původní síť existuje (získáme jej přičtením $l(e)$ zpět k nalezenému toku).

- **Chinese Postman Problem jako Tok s minimálními náklady:** Zadání požaduje projít každou hranu aspoň jednou s minimální délkou. Problém namodelujeme jako hledání cir-

kulace: dolní mez každé hrany $l(e) = 1$, horní $u(e) = \infty$, cena $c(e)$ je délka ulice. Bilance všech uzlů je 0.

- **Algoritmus rušení cyklů (Cycle Canceling) pro Min-Cost Flow:** Najdeme libovolný přípustný tok. Sestrojíme reziduální graf a pomocí Bellman-Fordova algoritmu v něm hledáme **cykly se zápornou celkovou cenou** (dopředné hrany mají cenu $c(e)$, zpětné $-c(e)$). Proženeme jím maximum možného toku. Opakujeme, dokud záporné cykly existují.
- **Maďarský algoritmus (Hungarian Alg.) pro Perfektní párování s min. váhou:** Bipartitní graf. Místo nákladů c_{ij} počítáme *transformované náklady* $c^p_{ij} = c_{ij} - p^x_i - p^y_j$, kde p jsou ohodnocení (potenciály) uzlů. Hledáme perfektní párování v *grafu rovnosti* (kde $c^p_{ij} = 0$). Pokud neexistuje, upravujeme potenciály p o minimální nenulovou hodnotu d.

4 Problém batohu (Knapsack)

Jsou dány předměty s cenami c_i a vahami w_i. Kapacita batohu je W.

- **Frakcionální batoh:** Předměty lze dělit. Optimální řešení najdeme seřazením podle poměru c_i/w_i a postupným plněním. Lze řešit polynomiálně $O(n \log n)$.
- **0/1 Batoh:** Předměty dělit nelze. NP-těžký problém. Řeší se dynamickým programováním.

Zkouška: DP tabulka a Aproximace

- **Dynamické programování ($O(nW)$):** Tabulka $T[i, w]$ udává maximální cenu z prvních i předmětů při kapacitě w.

$$T[i, w] = \max\big(T[i-1, w], \quad T[i-1, w-w_i] + c_i\big)$$

- **Zpětná rekonstrukce a Unikátnost:** Po vyplnění tabulky zjišťujeme, zda byl předmět vybrán. Začneme vpravo dole. Pokud $T[i, w] == T[i-1, w]$, *můžeme* předmět vynechat. Pokud se ale zároveň hodnota rovná i výrazu s přidáním předmětu ($T[i-1, w-w_i] + c_i$), máme na výběr dvě nezávislé cesty k optimální ceně. V takovém případě **řešení není unikátní**.
- **Složitostní chyták:** Algoritmus DP má složitost $O(n \cdot W)$. Je tedy pseudopolynomiální. Zkoušková otázka: *Co se stane, pokud víme, že $W \leq 10n$?*
  Odpověď: Složitost se stane $O(n \cdot 10n) = O(n^2)$. Protože složitost je nyní omezena polynomem závislým čistě na velikosti vstupu n, stává se z něj pro tuto podmnožinu instancí **striktně polynomiální** algoritmus.
- **2-Aproximace pro 0/1 Batoh:** Frakcionální batoh (s odseknutou částí kritického předmětu h) poskytuje horní mez optima. Z toho lze odvodit algoritmus: Vezmeme lepší ze dvou možností: a) množina předmětů $\{1 \dots h-1\}$, b) samotný předmět h. Důkaz: Součet cen těchto dvou řešení je větší nebo roven frakcionálnímu optimu, a tedy i reálnému optimu. Lepší z nich musí mít aspoň polovinu této sumy.
- **FPTAS (Fully Polynomial-Time Approx. Scheme):** 0/1 Batoh je jeden z mála problémů, který má FPTAS. Postup: Zvolíme chybu ϵ. Vydělíme všechny ceny faktorem $t = \frac{\epsilon \cdot c(S_1)}{n}$ (kde $c(S_1)$ je hodnota z 2-aproximace) a zaokrouhlíme dolů. Spustíme DP na těchto uměle snížených cenách. Složitost klesne na $O(n^2 \frac{1}{\epsilon})$.

5 Problém obchodního cestujícího (TSP)

Hledáme nejkratší Hamiltonovskou kružnici. Silně NP-těžký problém. Pro **Metrický TSP** platí trojúhelníková nerovnost $c(i, j) + c(j, k) \geq c(i, k)$ (tzn. zkratka nemůže prodloužit trasu).

5.1 Aproximační algoritmy (pro Metrický TSP)

- **Algoritmus dvojitého stromu (Double-Tree):** 1. Najdeme minimální kostru (MST). 2. Zdvojíme všechny hrany. 3. Vytvoříme Eulerovský tah. 4. Odstraníme opakující se uzly (zkratky). **Aproximační faktor: 2.**
- **Christofidův algoritmus:** 1. Najdeme MST. 2. Vybereme uzly s lichým stupněm v MST. 3. Pro tyto uzly najdeme perfektní párování s minimální váhou na úplném podgrafu. 4. Sjednotíme MST a párování (získáme multigraf, kde mají všechny uzly sudý stupeň). 5. Vytvoříme Eulerovský tah. 6. Provedeme zkratky. **Aproximační faktor: 3/2.**
- **Lokální vyhledávání (k-OPT):** Z existující trasy odebereme k hran a propojíme uzly jinak. Např. 2-OPT reverzuje pořadí průjezdu podgrafem.

Zkouška: Důkazy z TSP

- **Důkaz aproximačního faktoru 3/2 u Christofidesova algoritmu:**
  1. Nechť optimální řešení TSP má délku OPT. Odstraněním jedné hrany získáme kostru, tedy pro minimální kostru platí: $c(MST) \leq OPT$.
  2. Množina uzlů s lichým stupněm W tvoří úplný podgraf. Pokud vezmeme optimální TSP trasu a provedeme "zkratky"tak, aby procházela pouze uzly z W, získáme sub-tour s délkou $L_W \leq OPT$ (díky trojúhelníkové nerovnosti).
  3. Tuto sub-tour s lichým počtem hran lze rozdělit na dvě disjunktní perfektní párování M_1 a M_2. Platí $c(M_1) + c(M_2) = L_W \leq OPT$.
  4. Christofidův algoritmus vybírá párování s absolutně minimální vahou M. Tedy $c(M) \leq \min(c(M_1), c(M_2)) \leq \frac{1}{2}OPT$.
  5. Eulerovský tah L sjednotí MST a M. Jeho délka je $c(L) = c(MST) + c(M) \leq OPT + \frac{1}{2}OPT = \frac{3}{2}OPT$.
  6. Výsledná kružnice H vznikne zkratkováním L. Díky trojúhelníkové nerovnosti $c(H) \leq c(L) \leq \frac{3}{2}OPT$.
- **Důkaz neexistence r-aproximace pro obecný TSP:**
  Budeme postupovat sporem. Předpokládejme, že existuje polynomiální r-aproximační algoritmus pro obecný TSP. Ukážeme, že by se s ním dal řešit NP-úplný problém Hamiltonovské kružnice (HC) v polynomiálním čase (což nelze, leda by P=NP).
  Mějme graf G, ve kterém hledáme HC. Vytvoříme instanci TSP na úplném grafu:
  - Pokud hrana $e \in G$, její váha $c(e) = 1$.
  - Pokud hrana $e \notin G$, její váha $c(e) = r \cdot n + 1$ (vysoká penalizace).

  Pokud v G existuje HC, lze ji projít v našem TSP s délkou přesně n. Náš hypotetický r-aproximační algoritmus musí najít trasu s délkou $\leq r \cdot n$. Pokud v G HC neexistuje, **každá** TSP trasa musí použít alespoň jednu penalizační hranu a její celková délka bude $\geq (n-1) \cdot 1 + 1 \cdot (r \cdot n + 1) = r \cdot n + n > r \cdot n$.
  Algoritmus by tedy jasně odlišil existenci a neexistenci HC, čímž bychom řešili NP-úplný problém polynomiálně.

6 Rozvrhování (Scheduling)

Klasifikace pomocí Grahamovy notace $\alpha|\beta|\gamma$ (Typ/počet zdrojů — Omezení úloh — Optimalizační kritérium). Základní veličiny: p_j (doba zpracování), r_j (čas uvolnění), d_j (termín - due date), $\tilde{d}_j$ (striktní termín - deadline), C_j (čas dokončení). $L_j = C_j - d_j$ (zpoždění, lateness), $L_{max} = \max(L_j)$.

6.1 Algoritmy na jednom zdroji

- $1||L_{max}$ **(Pravidlo EDD - Earliest Due Date):** Seřadíme úlohy vzestupně podle d_j. Jednoduché a optimální. (Složitost $O(n \log n)$ dána tříděním).
- $1|pmtn, r_j|L_{max}$ **(Hornův algoritmus):** Preemptivní (přerušitelné) EDD. V každém okamžiku z dostupných úloh zpracováváme tu s nejmenším d_j.
- $1|prec|\sum w_j C_j$ **(Bratleyho typ B&B):** Hledá optimum přes větvící strom. Omezování se provádí pomocí LP relaxace nebo zjišťováním **BRTP (Block with Release Time Property)**. BRTP je spojitý blok úloh bez čekání, kde první spuštěná úloha má ze všech úloh v bloku nejmenší r_j. Pokud nalezneme rozvrh, který končí BRTP blokem, je tento rozvrh **garantovaně optimální**.

6.2 Algoritmy na paralelních identických zdrojích (P)

- $P|pmtn|C_{max}$ **(McNaughton):** Teoretické minimum $C^*_{max} = \max\left(\max p_i, \frac{\sum p_i}{R}\right)$. Úlohy plníme postupně do procesorů; pokud úloha přesáhne C^*_{max}, přerušíme ji a zbytek dáme na začátek dalšího procesoru.
- $P|pmtn, prec|C_{max}$ **(Muntz & Coffman - Level algoritmus):** Definujeme level úlohy jako délku nejdelší cesty z úlohy do listu grafu závislostí. Procesory přiřazujeme úlohám s nejvyšším levelem. Pokud je K dostupných úloh na nejvyšší úrovni a jen R procesorů (kde $K > R$), každá z těchto úloh běží poměrnou rychlostí $\beta = R/K$. Parametry (čas/rychlost) se přepočítávají, jakmile nějaká úloha skončí, nebo její level klesne na úroveň jiné čekající úlohy.

Zkouška: Pokročilé algoritmy a ILP Modely

- **Chetto-Silly-Bouchentouf** $(1|pmtn, prec, r_j, d_j|L_{max})$**:** Při výpočtu zkouškového příkladu algoritmus odstraňuje hrany závislostí úpravou časových oken úloh:
  1. *Dopředný chod (úprava r_j od zdrojů k listům):* $r'_j = \max\left(r_j, \max_{k \prec j}(r'_k + p_k)\right)$. Tím se zajistí, že následovník nezačne dřív, než fyzicky může skončit jeho předchůdce.
  2. *Zpětný chod (úprava d_j od listů ke zdrojům):* $d'_j = \min\left(d_j, \min_{k \succ j}(d'_k - p_k)\right)$. Tím se zajistí, že předchůdce skončí včas, aby následovník stihl svůj vlastní deadline.
  3. Výsledné úlohy jsou nezávislé. Spustíme na ně **Hornův algoritmus** s upravenými časy r' a d'.
- **ILP modely pro rozvrhování (Time-indexed vs. Relative order):**
  - **Time-indexed:** Zavedeme proměnnou $x_{it} \in \{0, 1\}$, kde $x_{it} = 1$ znamená, že úloha i začíná v čase t. Model vyžaduje diskrétní čas a jeho velikost závisí na teoretické horní mezi délky rozvrhu UB (počet proměnných i omezení roste s $O(n \cdot UB)$).
    - Každá úloha musí začít: $\sum_{t=0}^{UB-1} x_{it} = 1$.
    - Kapacita zdroje (v každý čas τ může běžet max 1 úloha): $\sum_i \sum_{t=\max(0,\tau-p_i+1)}^{\tau} x_{it} \leq 1$.
  - **Relative order (Relativní pořadí):** Proměnná $x_{ij} \in \{0, 1\}$ určuje, zda i předchází j. Spojitá proměnná s_i je přesný čas začátku úlohy. Omezení proti překryvu: $s_i + p_i \leq s_j + UB(1 - x_{ij})$. Velikost modelu nezávisí na délce času.
  - *Zkoušková otázka:* Jak se dá využít LP řešení relative order modelu uvnitř metody Branch & Bound?
    Odpověď: Relaxujeme proměnnou celočíselnosti $x_{ij} \in [0, 1]$. Tím získáme LP problém řešitelný polynomiálně. Hodnota účelové funkce tohoto LP nám poskytne **dolní mez**

(lower bound) pro zbývající práci v daném uzlu B&B stromu, kterou můžeme použít pro **pruning (ořezávání)** neperspektivních větví.

7 Constraint Programming (CSP)

CSP se na rozdíl od ILP nesnaží najít optimum primárně přes lineární rovnice, ale přes aktivní zmenšování domén (množin povolených hodnot) a prohledávání stromu. Model je tvořen množinou proměnných X, domén D a omezujících podmínek (constraints) C.

7.1 Úrovně konzistence (Propagace)

- **Hranná konzistence (Arc Consistency - AC):** Orientovaný oblouk (x_i, x_j) je AC, pokud pro **každou** hodnotu v D_i existuje aspoň jedna podpora v D_j, splňující omezení.
- **Zobecněná hranná konzistence (GAC):** Rozšíření AC pro n-ární omezení (omezení svazující více než 2 proměnné).
- **Konzistence mezí (Bounds Consistency - BC):** Rychlejší a slabší forma. Podpora se ověřuje pouze pro minimum a maximum číselného intervalu domény.
- **Cestní konzistence (Path Consistency - PC):** Ověřuje konzistenci na dvojicích proměnných vzhledem k třetí mezi-proměnné. Výpočetně velmi drahé.

7.2 Vyhledávací algoritmy (Search)

- **Chronologický návrat (Backtracking):** Přiřazuje hodnoty a kontroluje omezení až ex-post (když jsou proměnné plně instancovány). Velmi pomalé.
- **Forward Checking (Dopředná kontrola):** Po přiřazení hodnoty se aplikuje omezení a prořežou se domény pouze u *nepřiřazených* proměnných spojených s aktuální volbou.
- **MAC (Maintaining Arc Consistency):** Po každém rozhodnutí ve stromu se provede *kompletní propagace* na všech proměnných (např. spuštěním AC-3). Dramaticky prořezává větve stromu dřív, než do nich hledání vůbec vstoupí.

Zkouška: Algoritmus AC-3 a Globální omezení

- **Simulace algoritmu AC-3 (Časté zadání):** Dostanete proměnné, domény a množinu omezení a musíte ukázat kroky AC-3.
  1. Inicializace fronty Q: Vložíme všechny orientované oblouky. Pozor, omezení $x_1 \neq x_2$ generuje vždy **dva** oblouky: (x_1, x_2) a (x_2, x_1).
  2. `while` Q není prázdná: odebereme oblouk (x_i, x_j).
  3. Provedeme revizi `REVISE`(x_i, x_j): Pro každou hodnotu $v \in D_i$ hledáme aspoň jednu podporu $w \in D_j$. Pokud podpora neexistuje, hodnotu v odstraníme z D_i.
  4. **Kritický krok:** Pokud došlo ke změně domény D_i, musíme do Q **znovu přidat všechny oblouky** směřující *do* x_i, tzn. všechny hrany tvaru (x_k, x_i) pro $k \neq j$. Je to proto, že odstraněním hodnot z D_i jsme mohli zneplatnit podporu pro nějakou hodnotu v doméně souseda x_k.
- **Globální omezení `alldifferent`:** Proč nestačí napsat sérii binárních nerovností $x_i \neq x_j$? Představme si $x_1, x_2, x_3 \in \{1, 2\}$. Párová hranná konzistence neodhalí spor, protože pro $x_1 = 1$ existuje podpora $x_2 = 2$ atd. Globální omezení `alldifferent` řeší závislost komplexně: interně vytvoří **bipartitní graf** (Proměnné na jedné straně, Hodnoty na druhé). Uvnitř se aplikuje **algoritmus hledání maximálního párování**. Ten okamžitě zjistí, že 3 proměnné

nelze spárovat s pouhými 2 hodnotami, a okamžitě vyhodnotí stav jako neřešitelný (prázdná doména), čímž vynutí backtrack.

- **Nogoods (Záznamy selhání):** Při backtrackingu si solver může vygenerovat a uložit "Nogood" – logickou klauzuli (např. $x_1 \neq 3 \vee x_2 \neq 4$), která říká, že tato konkrétní kombinace vedla do slepé uličky. Při restartu solveru se tato klauzule přidá jako nové omezení, takže solver už nikdy stejnou chybu nezopakuje.